

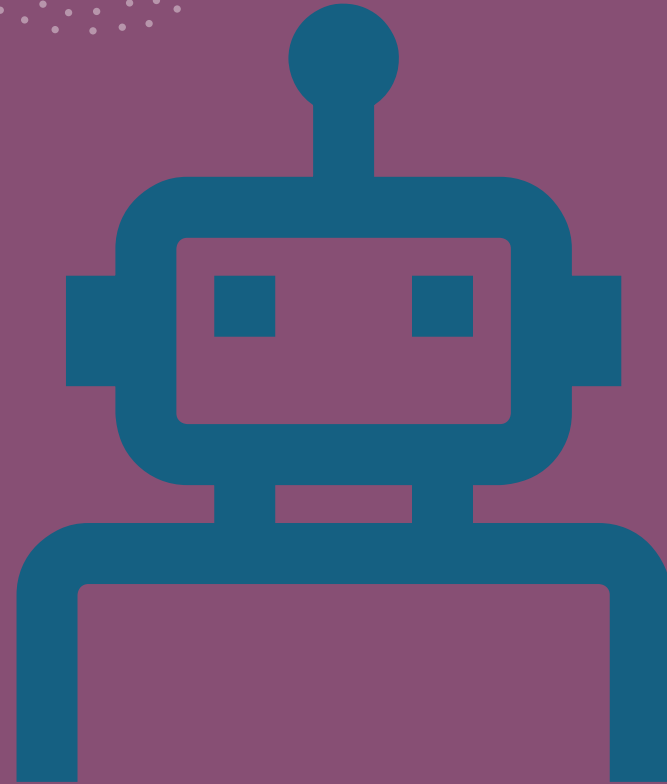
Gradient Checkpointing

- **Strategy:** Store fewer intermediates, recompute during backward
- **Trade-off:** $\sqrt{n}$ memory reduction for $\sim 33\%$ computation increase
- **Example:** Store every k -th layer, recompute the rest
- **Implementation:** Framework manages storage/recomputation automatically
- **Result:** Enables training of much larger models



TinyML Gradient Computation

- • **TinyML Characteristics:**
- Energy: $< 1 \text{ mW}$
- Memory: Kilobytes total
- No dynamic allocation
- Real-time requirements
 - **Challenge:** Wake word detection with federated learning
 - **Solution:** Aggressive checkpointing with full recomputation



Memory-Efficient AD Implementation

- **Micro-batching:** Process one example at a time
- **Quantized Gradients:** 16-bit or 8-bit gradient computation
- **Static Allocation:** Pre-allocate all memory at compile time
- **Template Optimization:** Generate specialized code for specific architectures
- **Result:** Complete AD system in <5KB binary

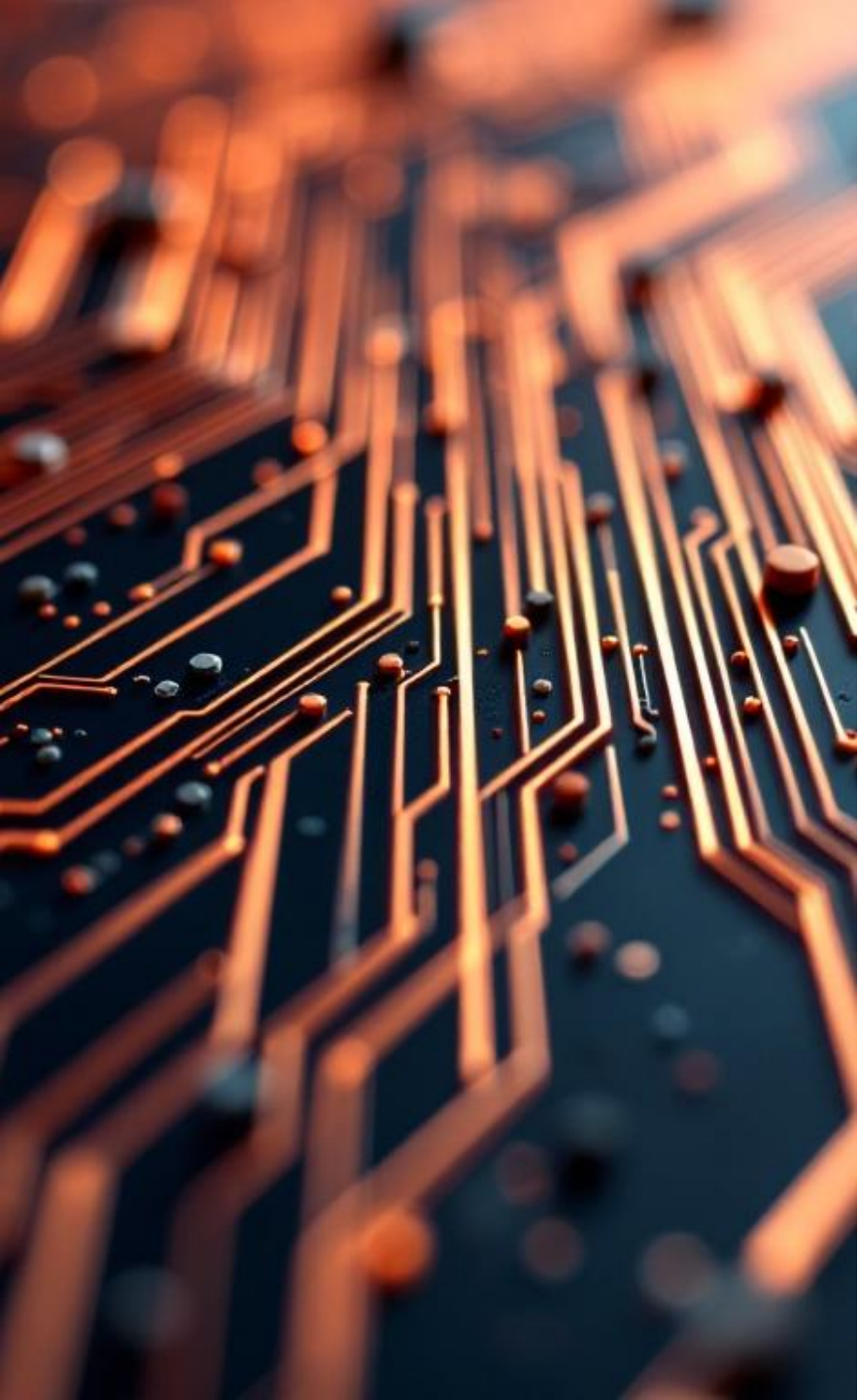
Federated Learning Case Study

Scenario: On-device wake word detection

Requirements:

- Local gradient computation on audio
- <32KB total RAM
- Real-time operation
- Federated participation
 - **Implementation:** Forward-only checkpointing + quantization
 - **Results:** 5-10x computation, fits constraints





Energy-Power Relationship

Fundamental Energy Concepts

Power vs. Energy:

- Power = Energy / Time (watts = joules/second)
- For battery life: Average power determines duration
- 1mW average → 2x longer life than 2mW average

Battery Capacity Examples:

- CR2032 coin battery: 2,500J (1 month at 1mW)
- AA battery: 15,000J (6 months at 1mW)
- Phone battery: ~50,000J

Energy Harvesting Potential:

- Indoor light: $10\mu\text{W}/\text{cm}^2$
- Outdoor light: $10\text{mW}/\text{cm}^2$
- Industrial thermal: $1\text{-}10\text{mW}/\text{cm}^2$
- Vibration: $10\text{-}100\mu\text{W}/\text{cm}^2$

Design Implication:

Only outdoor solar and industrial thermal currently practical for self-powered ML devices.

Duty Cycling Strategies

Trading Computation Time for Power

Basic Duty Cycling:

```
void low_power_inference_loop() {  
    while(1) {  
        // Active period  
        digitalWrite(LED_PIN, HIGH);  
        run_ml_inference();  
        process_results();  
  
        // Sleep period  
        digitalWrite(LED_PIN, LOW);  
        sleep_mode(SLEEP_DURATION);  
    }  
}
```

Power Reduction Calculation:

- Active: 10mW for 100ms every 1000ms
- Sleep: 0.1mW for 900ms every 1000ms
- Average: $(10 \times 0.1 + 0.1 \times 0.9) = 1.09\text{mW}$
- vs. continuous 10mW operation

Frequency Scaling:

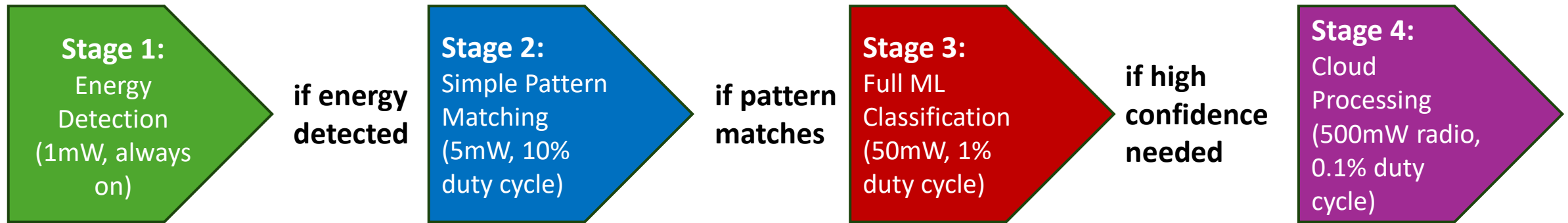
Lower clock frequency reduces power quadratically:

- 48MHz: 2mW
- 24MHz: 1.2mW
- 8MHz: 0.4mW

Trade computation speed for power savings

Cascading Design Principles

Multi-Stage Processing for Efficiency



Average Power Calculation:

- Stage 1: $1\text{mW} \times 100\% = 1.0\text{mW}$
- Stage 2: $5\text{mW} \times 10\% = 0.5\text{mW}$
- Stage 3: $50\text{mW} \times 1\% = 0.5\text{mW}$
- Stage 4: $500\text{mW} \times 0.1\% = 0.5\text{mW}$
- **Total: 2.5mW average**

Key Principles:

Each stage has higher accuracy but higher power
Later stages activate only when needed
False positive rate determines duty cycle

Energy-Aware Design Optimization

Minimizing Power Consumption

Power sources to watch

1. Computation energy

- 8-bit multiply $\approx$ 20 femtojoules
- 16-bit multiply $\approx$ 80 femtojoules
- 32-bit multiply $\approx$ 320 femtojoules
- Memory read/write: roughly 5–1000 picojoules depending on the level

2. Memory access energy

- Register access is cheapest
- Cache access costs a few to tens of picojoules
- DRAM access is most expensive: 1–10 nanojoules

3. Peripheral energy

- Sensor sampling: microjoules per sample
- Radio transmission: millijoules per packet

Optimization strategies

Reduce computation

- Prefer bit shifts and adds over multiplies when mathematically equivalent
- Use lower precision (int8/int16) where accuracy allows
- Fuse operations and reuse intermediate results

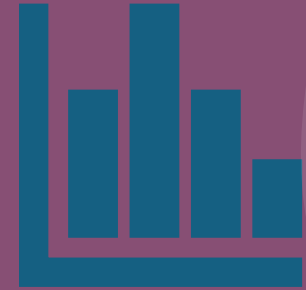
Minimize memory access

- Process data in place to avoid copies
- Keep hot data in registers or on-chip SRAM; avoid DRAM
- Arrange data for cache-friendly, sequential access
- Use fixed-size buffers and avoid dynamic allocation

Duty cycling

- Sleep between capture and inference cycles
- Batch work: wake briefly, process, then return to deep sleep
- Keep radios off; transmit only summaries or alerts at low duty cycle

Performance Validation and Benchmarking



Measuring real-world performance - Benchmarking methodology

1) Accuracy validation

- Compare the optimized model against a baseline on a representative test set. Report final accuracy and the drop from baseline. Target is minimal loss while gaining efficiency.

2) Performance measurement

- Measure end-to-end inference time on the actual device. Use timestamps around the full pipeline and report average, p50, p90, and worst-case latency over many runs.

3) Energy measurement

- Profile current draw during active processing and during sleep. Multiply by supply voltage to get power, then integrate over time to estimate energy per inference and average current at your duty cycle. Include radio and sensor activity if used.

Validation criteria

- Accuracy: less than 2% absolute degradation from baseline
- Latency: meets real-time requirements with margin
- Energy: fits the battery-life budget for the intended duty cycle
- Memory: stays within RAM and flash limits with headroom